

Progetto di paradigmi di programmazione e sviluppo

Testing automatizzato di interfacce grafiche

I. Introduzione

Negli ultimi decenni le interfacce grafiche hanno assunto un ruolo centrale nello sviluppo del software. La maggior parte delle applicazioni moderne, sia desktop che web o mobile, viene utilizzata attraverso una Graphical User Interface (GUI), che rappresenta il principale punto di contatto tra l'utente e il sistema. Di conseguenza, la qualità dell'interfaccia grafica influisce direttamente sull'esperienza dell'utente e sulla percezione complessiva dell'affidabilità del software.

Tradizionalmente il collaudo delle interfacce grafiche è eseguito manualmente da sviluppatori, tester o utenti finali. Sebbene tale approccio permetta di individuare numerosi difetti, esso presenta diversi limiti: richiede tempo, è costoso, è difficilmente ripetibile e può essere soggetto a errori umani. Inoltre, all'aumentare della complessità dell'applicazione, il numero di possibili interazioni cresce rapidamente, rendendo impraticabile una verifica completamente manuale. Per affrontare tali problematiche sono stati sviluppati strumenti e metodologie che consentono di automatizzare il testing delle interfacce grafiche. Il GUI Testing automatizzato consiste nell'utilizzo di software in grado di simulare le azioni di un utente reale, come click del mouse, inserimento di testo o selezione di elementi dell'interfaccia, e di verificare automaticamente che il comportamento osservato corrisponda a quello atteso.

Tuttavia, il testing automatico delle GUI presenta difficoltà specifiche che lo distinguono da altre forme di testing software. Le interfacce grafiche sono infatti sistemi fortemente orientati agli eventi, caratterizzati da uno stato interno che evolve nel tempo e da possibili comportamenti asincroni. Tali caratteristiche rendono più complessa la definizione di casi di test affidabili e facilmente manutenibili.

L'obiettivo di questo progetto è analizzare come sia possibile automatizzare il testing delle interfacce grafiche, approfondendo le principali problematiche associate a questo processo e valutando l'utilizzo del framework TestFX per il collaudo di applicazioni JavaFX. A tal fine verrà sviluppata una semplice applicazione Todo List secondo il pattern architetturale Model-View-Controller (MVC), che sarà utilizzata come caso di studio per progettare ed eseguire una suite di test automatici. Attraverso questa sperimentazione sarà possibile valutare vantaggi, limiti e criticità dell'automazione del GUI Testing.

II. Fondamenti del Software Testing

1) Qualità del software

Si parte dalla necessità di produrre software di qualità. La qualità del software rappresenta la capacità di un sistema di soddisfare i requisiti funzionali e non funzionali per i quali è stato progettato. Un software di qualità deve fornire risultati corretti, essere affidabile, mantenibile, efficiente e facilmente utilizzabile dagli utenti.

Per garantire tali caratteristiche è necessario adottare procedure di verifica e validazione durante l'intero ciclo di sviluppo. La verifica (verification) mira a controllare che il software sia stato costruito correttamente rispetto alle specifiche progettuali mentre la validazione (validation), invece, verifica che il software soddisfi effettivamente le esigenze dell'utente finale. Il testing rappresenta uno degli strumenti principali utilizzati per svolgere entrambe queste attività.

2) Definizione di Software Testing

Il Software Testing può essere definito come il processo di esecuzione e valutazione di un sistema software con l'obiettivo di individuare difetti e verificare il corretto funzionamento delle sue componenti. Secondo una definizione classica proposta da Glenford Myers, il testing è il processo di esecuzione di un programma con l'intento di trovare errori.

L'obiettivo del testing non è dimostrare l'assenza di difetti, ma aumentare il livello di fiducia nella correttezza del sistema attraverso una verifica sistematica del suo comportamento. Per questo motivo, nel ciclo di sviluppo software esistono diversi livelli di testing, ciascuno focalizzato su specifici aspetti del sistema.

a) Unit Testing

Gli unit test verificano il corretto funzionamento delle singole unità di codice, come metodi o classi. L'obiettivo è isolare il componente da testare e verificare che produca i risultati attesi per determinati input. Per esempio, verificare che un metodo di somma restituisca correttamente il risultato atteso per una determinata coppia di operandi.

b) Integration Testing

L'integration testing verifica la corretta interazione tra più componenti del sistema.

In questa fase si controlla che moduli sviluppati separatamente collaborino correttamente tra loro, scambiandosi dati e servizi nel modo previsto.

c) System Testing

Il system testing considera l'applicazione come un sistema completo e verifica il rispetto dei requisiti funzionali e non funzionali.

Questo livello di testing consente di individuare problemi che potrebbero non emergere durante il collaudo dei singoli componenti.

d) Acceptance Testing

L'acceptance testing rappresenta l'ultima fase di verifica prima del rilascio del software. Lo scopo è determinare se il sistema soddisfa i requisiti richiesti dal cliente o dagli utenti finali.

3) Posizionamento del GUI Testing

Il GUI Testing si colloca principalmente tra il system testing e l'acceptance testing, poiché verifica il comportamento osservabile dell'applicazione dal punto di vista dell'utente.

A differenza degli unit test, che operano direttamente sul codice, il GUI Testing interagisce con l'interfaccia grafica simulando operazioni reali, come:

- click su pulsanti;
- inserimento di testo;
- selezione di checkbox;
- apertura di finestre di dialogo;
- navigazione tra schermate.

Questo approccio consente di verificare non solo la correttezza della logica applicativa, ma anche la corretta integrazione tra interfaccia utente e componenti sottostanti.

4) Perché automatizzare il testing?

L'automazione del testing nasce dall'esigenza di ridurre i costi e aumentare l'efficienza delle attività di verifica.

Tra i principali vantaggi dell'automazione si possono evidenziare:

- esecuzione rapida e ripetibile dei test;
- riduzione degli errori umani;
- maggiore copertura dei casi di test;
- individuazione precoce delle regressioni.

Tuttavia, l'automazione non rappresenta una soluzione universale. Lo sviluppo e la manutenzione delle suite di test comportano infatti costi aggiuntivi che devono essere giustificati dai benefici ottenuti. Nel caso delle interfacce grafiche, questi aspetti assumono particolare rilevanza a causa della natura dinamica e fortemente interattiva delle GUI. La comprensione delle specifiche problematiche associate al GUI Testing costituisce quindi un passaggio fondamentale per valutare quando e come l'automazione possa risultare efficace.

III. Il GUI Testing

1) Definizione di GUI Testing

Il Graphical User Interface Testing (GUI Testing) è una tecnica di verifica software finalizzata a controllare il corretto funzionamento dell'interfaccia grafica di un'applicazione. A differenza degli unit test, che verificano il comportamento di singoli componenti software in isolamento, il GUI Testing osserva il sistema dal punto di vista dell'utente finale, simulando interazioni reali e verificando che l'interfaccia reagisca correttamente.

L'obiettivo principale consiste nel garantire che tutti gli elementi dell'interfaccia siano in grado di svolgere le funzioni per cui sono stati progettati e che il comportamento complessivo dell'applicazione sia coerente con i requisiti definiti durante la fase di progettazione.

Tra gli elementi tipicamente sottoposti a verifica si trovano:

- pulsanti;
- campi di testo;
- checkbox;
- menu;
- finestre di dialogo;
- liste e tabelle;
- messaggi informativi e di errore;
- componenti di navigazione.

Il GUI Testing rappresenta uno dei livelli di testing più vicini all'esperienza reale dell'utente e consente di individuare difetti che potrebbero non emergere attraverso altre tecniche di verifica.

2) Interazione tra utente e sistema

Le applicazioni con interfaccia grafica sono progettate per reagire alle azioni dell'utente. Quando un utente interagisce con un elemento dell'interfaccia, viene generato un evento che viene intercettato dal sistema e gestito attraverso specifici meccanismi software.

Ad esempio, il click su un pulsante può generare la seguente sequenza:

Utente → Click → Evento → Event Handler → Aggiornamento dello stato → Aggiornamento della GUI

Questo modello di funzionamento differisce significativamente dall'esecuzione lineare di un programma tradizionale. Nel GUI Testing è quindi necessario simulare tali interazioni e verificare che ogni evento produca gli effetti attesi sul sistema.

3) Interfacce grafiche come sistemi event-driven

La maggior parte delle moderne interfacce grafiche è basata sul paradigma Event-Driven Programming. In questo paradigma il flusso di esecuzione dell'applicazione non è determinato da una sequenza predefinita di istruzioni, ma dagli eventi generati dagli utenti o dal sistema.

Nel caso di JavaFX, ad esempio, un pulsante può essere associato a un gestore di eventi:

`button.setOnAction(event -> addTask());`

Quando l'utente preme il pulsante, viene generato un evento che provoca l'esecuzione del metodo associato. Questo approccio aumenta la flessibilità delle applicazioni ma introduce nuove difficoltà durante il testing, poiché il comportamento del sistema dipende dalle sequenze di eventi generate durante l'esecuzione.

4) Verifica del comportamento dell'interfaccia

Durante il GUI Testing non è sufficiente verificare che un evento venga generato correttamente. Occorre anche controllare che il sistema produca gli effetti previsti. Ad esempio, in una applicazione Todo List, dopo l'inserimento di una nuova attività e la pressione del pulsante di aggiunta, il sistema dovrebbe:

1. acquisire il testo inserito;
2. creare una nuova attività;
3. aggiornare la lista visualizzata;
4. aggiornare il contatore delle attività;
5. mostrare un messaggio di conferma.

Il test dovrà quindi verificare sia l'esecuzione dell'azione sia le conseguenze osservabili sull'interfaccia.

5) Automazione del GUI Testing

Il GUI Testing può essere svolto manualmente oppure automaticamente. Nel testing manuale un operatore esegue direttamente le azioni previste dai casi di test e verifica visivamente i risultati. Nel testing automatizzato, invece, un framework software esegue automaticamente le stesse operazioni simulando il comportamento di un utente reale.

Le principali attività automatizzabili sono:

- click del mouse;
- pressione di tasti;
- inserimento di testo;
- selezione di elementi;
- apertura e chiusura di finestre;
- verifica di contenuti visualizzati.

L'automazione consente di eseguire rapidamente gli stessi test più volte, riducendo il rischio di errori umani e migliorando la ripetibilità delle verifiche.

6) Benefici del GUI Testing automatizzato

L'utilizzo di strumenti di automazione offre numerosi vantaggi tra cui:

- **Ripetibilità:** I test possono essere eseguiti molte volte ottenendo sempre le stesse condizioni operative.
- **Riduzione dei costi:** Dopo l'investimento iniziale necessario per sviluppare la suite di test, le verifiche possono essere eseguite automaticamente senza richiedere intervento umano.
- **Individuazione delle regressioni:** Ogni modifica al software può introdurre nuovi difetti. Le suite di test automatici permettono di verificare rapidamente che funzionalità precedentemente corrette continuino a funzionare.
- **Integrazione continua:** I test possono essere eseguiti automaticamente durante il processo di build, favorendo pratiche moderne come Continuous Integration e Continuous Delivery.
- **Maggiore copertura:** L'automazione consente di eseguire numerosi scenari di test che sarebbero difficili da ripetere manualmente con la stessa accuratezza.

7) Limiti del GUI Testing automatizzato

Nonostante i numerosi vantaggi, il GUI Testing automatizzato presenta anche alcuni limiti:

- La progettazione e la manutenzione dei test richiedono tempo e competenze specifiche.
- Inoltre, le modifiche all'interfaccia grafica possono rendere obsoleti i test esistenti, aumentando i costi di manutenzione.
- Alcuni aspetti, come l'usabilità, l'accessibilità o la qualità estetica dell'interfaccia, risultano difficilmente valutabili attraverso procedure completamente automatizzate. Per questo motivo il testing automatico viene generalmente considerato complementare e non sostitutivo del testing manuale.

8) Il ruolo del GUI Testing nel progetto

Nel contesto di questo progetto il GUI Testing verrà analizzato attraverso lo sviluppo di una applicazione Todo List realizzata con JavaFX. L'automazione dei test sarà affidata al framework TestFX, che consentirà di simulare le interazioni dell'utente e verificare automaticamente il comportamento dell'interfaccia.

Questo caso di studio permetterà di valutare concretamente le opportunità e le difficoltà associate all'automazione del testing delle interfacce grafiche.

iv. Problematiche del GUI Testing

Nonostante i numerosi vantaggi offerti dall'automazione, il testing delle interfacce grafiche rappresenta una delle attività più complesse nell'ambito della verifica del software. Le GUI costituiscono sistemi dinamici, interattivi e fortemente dipendenti dal contesto di esecuzione.

Per queste ragioni il GUI Testing presenta problematiche specifiche che non emergono con la stessa intensità in altre forme di testing.

1) Gestione dello stato dell'interfaccia

Un'interfaccia grafica può essere considerata come un sistema dotato di stato. Lo stato rappresenta l'insieme delle informazioni che descrivono la situazione corrente dell'applicazione.

Nel caso della Todo List sviluppata in questo progetto, lo stato comprende:

- l'insieme delle attività presenti;
- lo stato di completamento delle attività;
- il filtro attualmente selezionato;
- il contenuto dei campi di input;
- le informazioni mostrate all'utente.

Ogni azione dell'utente provoca una transizione da uno stato a un altro.

Ad esempio:

- S0 → Lista vuota
- S1 → Attività aggiunta
- S2 → Attività completata
- S3 → Attività eliminata

Il testing deve verificare che tutte le transizioni avvengano correttamente e che lo stato dell'interfaccia rimanga sempre coerente.

2) Esplosione combinatoria delle interazioni

Uno dei principali problemi del GUI Testing è rappresentato dalla crescita esponenziale delle possibili sequenze di eventi. Anche un'interfaccia relativamente semplice può contenere numerosi controlli interattivi.

Se un'applicazione possiede dieci pulsanti e ciascuno può essere premuto in diversi momenti, il numero delle possibili sequenze cresce rapidamente. Di conseguenza risulta impossibile testare esaustivamente tutte le combinazioni di interazione.

Per affrontare questo problema vengono generalmente selezionati scenari rappresentativi che coprano i comportamenti più significativi dell'applicazione. La scelta dei casi di test rappresenta quindi un'attività nella progettazione di una suite di GUI Testing.

3) Asincronia e sincronizzazione

Le moderne interfacce grafiche eseguono numerose operazioni in modo asincrono.

Nel caso di JavaFX, ad esempio, gli aggiornamenti dell'interfaccia vengono gestiti attraverso il JavaFX Application Thread. Questo significa che alcune modifiche visive potrebbero non essere immediatamente disponibili dopo l'esecuzione di un'azione. Un test potrebbe quindi verificare lo stato dell'interfaccia prima che quest'ultima abbia completato il proprio aggiornamento. Tale situazione può generare risultati incoerenti e rendere difficile la realizzazione di test affidabili. I framework di GUI Testing devono pertanto fornire meccanismi di sincronizzazione che consentano di attendere il completamento delle operazioni prima di eseguire le verifiche.

4) Oracle Problem

Uno dei problemi più noti nel campo del software testing è il cosiddetto Oracle Problem. Un test oracle è un meccanismo utilizzato per stabilire se il risultato prodotto da un programma sia corretto o meno. Nel caso degli unit test, l'oracolo è generalmente semplice da definire. Ad esempio, è immediato verificare che una funzione matematica restituisca il risultato atteso.

Nel GUI Testing la situazione è più complessa. Spesso non è sufficiente verificare un singolo valore, ma è necessario valutare lo stato complessivo dell'interfaccia. Ad esempio, dopo l'aggiunta di una nuova attività alla Todo List, il sistema dovrebbe:

- aggiornare la lista;
- aggiornare il contatore;
- mantenere coerente il filtro selezionato;
- mostrare un messaggio appropriato.

Definire automaticamente tutti questi criteri di correttezza rappresenta una delle principali difficoltà del GUI Testing.

5) Flaky Tests

I flaky tests sono test che producono risultati differenti in esecuzioni successive pur in assenza di modifiche al codice. In altre parole, un test può superare correttamente una verifica e fallire nella successiva senza che il comportamento dell'applicazione sia realmente cambiato.

Le cause più comuni includono:

- problemi di sincronizzazione;
- dipendenze dall'ambiente di esecuzione;
- temporizzazioni non deterministiche;
- accesso concorrente alle risorse;
- differenze tra sistemi operativi.

I flaky tests rappresentano un problema particolarmente rilevante nel GUI Testing poiché riducono la fiducia degli sviluppatori nella validità della suite di test.

6) Fragilità e manutenzione dei test

Le interfacce grafiche sono soggette a frequenti modifiche durante il ciclo di vita del software. Anche cambiamenti apparentemente minimi possono influenzare il funzionamento dei test automatici. Ad esempio:

- modifica del testo di un pulsante;
- rinomina di un identificatore;
- cambiamento della struttura della schermata;
- spostamento di componenti.

Per limitare questi problemi è buona pratica utilizzare identificatori stabili e progettare i test in modo indipendente dall'aspetto grafico quando possibile. Nonostante queste precauzioni, la manutenzione delle suite di GUI Testing rappresenta spesso una parte significativa del costo complessivo dell'automazione.

7) Bilanciamento tra costi e benefici

L'automazione del GUI Testing offre numerosi vantaggi, ma richiede un investimento iniziale significativo. Lo sviluppo dei test, la configurazione dell'ambiente e la manutenzione della suite comportano costi che devono essere giustificati dai benefici ottenuti.

In generale, l'automazione risulta particolarmente conveniente quando:

- il software evolve frequentemente;
- i test devono essere eseguiti ripetutamente;
- il numero di casi di test è elevato;
- è necessario integrare i controlli nei processi di Continuous Integration.

Al contrario, per applicazioni molto piccole o soggette a modifiche continue dell'interfaccia, i costi di manutenzione possono ridurre l'efficacia dell'approccio automatizzato.

V. Tecniche di Automazione del GUI Testing

Prima della diffusione degli strumenti di automazione, il collaudo delle interfacce grafiche veniva eseguito quasi esclusivamente in modo manuale. In questo approccio un tester utilizza direttamente l'applicazione, eseguendo una serie di operazioni e verificando visivamente il comportamento del sistema.

Ad esempio, nel caso di una Todo List, il tester potrebbe:

- inserire una nuova attività;
- premere il pulsante di aggiunta;
- verificare che la nuova attività compaia nella lista.

Vantaggi

- elevata flessibilità;
- capacità di valutare aspetti soggettivi come usabilità ed esperienza utente;
- nessuna necessità di sviluppare codice di test.

Svantaggi

- elevato costo in termini di tempo;
- difficoltà di ripetizione;
- maggiore probabilità di errori umani;
- scarsa scalabilità.

Per questi motivi il testing manuale viene spesso affiancato da tecniche automatizzate. L'automazione del GUI Testing può essere realizzata attraverso diverse tecniche, sviluppate nel tempo per affrontare le problematiche associate alla verifica delle interfacce grafiche. Non esiste un approccio universalmente migliore degli altri. Ogni tecnica presenta vantaggi e svantaggi e risulta più o meno adatta a seconda del contesto applicativo, della complessità dell'interfaccia e degli obiettivi del processo di testing.

Di seguito presentiamo i principali approcci utilizzati nell'automazione del GUI Testing, evidenziandone caratteristiche, benefici e limitazioni.

1) Capture and Replay

Una delle prime strategie sviluppate per l'automazione delle GUI è il cosiddetto approccio Capture and Replay. In questo modello uno strumento registra le azioni eseguite da un utente durante una sessione di utilizzo dell'applicazione. Successivamente la stessa sequenza può essere riprodotta automaticamente.

Il processo è generalmente composto da due fasi:

- registrazione delle interazioni;
- riproduzione automatica.

Vantaggi

- facilità di utilizzo;
- nessuna necessità di conoscenze di programmazione;
- rapida creazione di test.

Svantaggi

- elevata fragilità;
- scarsa manutenibilità;
- difficoltà di adattamento ai cambiamenti dell'interfaccia.

Per tali motivi questo approccio è oggi utilizzato principalmente in contesti limitati.

2) Script-Based Testing

Lo Script-Based Testing rappresenta attualmente uno degli approcci più diffusi. In questo caso i test vengono scritti direttamente sotto forma di codice. Le interazioni dell'utente vengono simulate attraverso istruzioni che identificano gli elementi dell'interfaccia e ne riproducono il comportamento.

Ad esempio:

- click su un pulsante;
- inserimento di testo;
- selezione di una checkbox;
- verifica di un messaggio.

Framework come Selenium, Playwright e TestFX appartengono a questa categoria.

Vantaggi

- elevata flessibilità;
- buona manutenibilità;
- integrazione con framework di testing;
- possibilità di riutilizzo del codice.

Svantaggi

- maggiore complessità iniziale;
- necessità di competenze di programmazione;
- manutenzione della suite di test.

Nel presente progetto verrà adottato proprio questo approccio attraverso l'utilizzo di TestFX.

3) Model-Based Testing

Il Model-Based Testing è una tecnica avanzata che si basa sulla costruzione di un modello formale del comportamento dell'applicazione. L'interfaccia viene rappresentata come un insieme di stati e transizioni. Ogni azione dell'utente provoca il passaggio da uno stato a un altro.

Ad esempio, per una Todo List:

S0 → Lista vuota

S1 → Attività aggiunta

S2 → Attività completata

S3 → Attività eliminata

A partire da questo modello è possibile generare automaticamente casi di test.

Vantaggi

- elevata copertura;
- individuazione sistematica dei percorsi di esecuzione;
- riduzione dell'intervento manuale.

Svantaggi

- complessità nella costruzione del modello;
- costi elevati per sistemi di grandi dimensioni.

Questa tecnica è particolarmente interessante dal punto di vista teorico poiché collega il GUI Testing alla teoria degli automi e dei sistemi a stati.

4) Property-Based Testing

Nel Property-Based Testing non vengono definiti singoli casi di test, ma proprietà che devono essere sempre soddisfatte dal sistema. Ad esempio, nella Todo List si potrebbe definire la seguente proprietà:

"Dopo l'aggiunta di una nuova attività, il numero totale di attività deve aumentare di una unità."

Un framework genera automaticamente numerosi dati di input e verifica che la proprietà sia sempre rispettata.

Vantaggi

- capacità di individuare casi limite;
- elevata automazione;
- riduzione del bias umano nella definizione dei test.

Svantaggi

- difficoltà nella definizione delle proprietà;
- minore diffusione rispetto ad altre tecniche.

5) Confronto tra le tecniche

Le diverse tecniche presentano caratteristiche differenti. Il testing manuale offre flessibilità ma scarsa automazione. L'approccio Capture and Replay consente una rapida creazione dei test ma soffre di problemi di manutenzione. Lo Script-Based Testing richiede competenze di programmazione ma rappresenta oggi la soluzione più diffusa e versatile. Il Model-Based Testing e il Property-Based Testing offrono elevati livelli di automazione e copertura, ma introducono una maggiore complessità concettuale e implementativa.

La scelta della tecnica più appropriata dipende quindi dal contesto applicativo, dalle risorse disponibili e dagli obiettivi del processo di testing.

VI. Framework per il GUI Testing

L'automazione del GUI Testing richiede l'utilizzo di strumenti in grado di simulare le interazioni dell'utente e verificare il comportamento dell'interfaccia grafica. Nel corso degli anni sono stati sviluppati numerosi framework, differenziati per tecnologia supportata, modalità di interazione e livello di automazione offerto. La scelta dello strumento più appropriato dipende da diversi fattori, tra cui il tipo di applicazione da testare, il linguaggio di programmazione utilizzato e gli obiettivi del processo di verifica.

Adesso analizziamo alcuni dei framework più diffusi nel panorama del GUI Testing.

1) ScalaTest

ScalaTest è un framework di testing, che si occupa di organizzare i test, eseguirli, verificare i risultati (assert), generare report. Non ha funzionalità native per interagire con un'interfaccia grafica.

Per fare GUI testing in Scala, ScalaTest viene tipicamente combinato con un framework di automazione della GUI come **Selenium WebDriver** per applicazioni web.

Vantaggi

- Framework standard per il testing in Scala;
- Supporta diversi stili di testing;
- Facilmente integrabile con Selenium;

Svantaggi

- Non fornisce funzionalità di automazione della GUI.
- Necessita di un framework esterno per interagire con l'interfaccia grafica.

2) AssertJ Swing

AssertJ Swing è un framework progettato per il testing automatico di applicazioni Java Swing. Permette di simulare le interazioni dell'utente e verificare lo stato dei componenti dell'interfaccia.

Vantaggi

- integrazione con l'ecosistema Java;
- API semplici e intuitive.

Svantaggi

- limitato alle applicazioni Swing;
- non adatto alle applicazioni JavaFX.

Poiché il presente progetto utilizza JavaFX, AssertJ Swing non rappresenta una soluzione adeguata.

3) TestFX

TestFX è un framework open source specificamente progettato per il testing automatico di applicazioni JavaFX. Consente di simulare le principali interazioni dell'utente e di verificare automaticamente il comportamento dell'interfaccia.

TestFX si integra facilmente con JUnit e con gli strumenti di build più diffusi, come Maven e Gradle.

Vantaggi

- progettato specificamente per JavaFX;
- integrazione con JUnit;
- API intuitive;
- supporto alle verifiche automatiche della GUI;
- facilità di integrazione nei processi di Continuous Integration.

Svantaggi

- limitato all'ecosistema JavaFX;
- comunità più ridotta rispetto a framework come Selenium.

Nonostante tali limitazioni, TestFX rappresenta la soluzione più naturale per il testing automatico di applicazioni JavaFX.

VII. TestFX

TestFX è un framework open source progettato per il testing automatico di applicazioni sviluppate con JavaFX. L'obiettivo principale del framework è consentire agli sviluppatori di simulare le interazioni di un utente reale e verificare automaticamente il comportamento dell'interfaccia grafica. Questo approccio consente di verificare il comportamento osservabile dell'applicazione da una prospettiva molto vicina a quella dell'utente finale.

TestFX si basa sull'integrazione tra JavaFX e il framework di testing JUnit. Durante l'esecuzione di un test, il framework:

1. avvia l'applicazione JavaFX;
2. crea l'ambiente di test;
3. esegue le interazioni simulate;
4. verifica i risultati ottenuti;
5. chiude l'ambiente di esecuzione.

In questo modo è possibile riprodurre in maniera controllata il comportamento di un utente reale.

1) Identificazione dei componenti

Per interagire con i componenti della GUI, TestFX deve essere in grado di individuarli. Il metodo più comune consiste nell'assegnare un identificatore univoco ai controlli JavaFX:

```
inputField.setId("inputField");
```

Successivamente il componente può essere selezionato nel test:

```
robot.clickOn("#inputField");
```

L'utilizzo degli identificatori presenta diversi vantaggi:

- maggiore leggibilità;
- minore dipendenza dalla struttura grafica;
- migliore manutenibilità.

Per questo motivo rappresenta una buona pratica nella progettazione di applicazioni destinate al GUI Testing.

2) Verifica dello stato dell'interfaccia

Dopo aver simulato una sequenza di interazioni, è necessario verificare che l'applicazione abbia prodotto il comportamento atteso.

Le verifiche possono riguardare:

- contenuto di etichette;
- numero di elementi presenti in una lista;

- stato di checkbox;
- visibilità di componenti;
- contenuto di campi di testo.

3) Gestione della sincronizzazione

Come discusso nei capitoli precedenti, le applicazioni GUI sono spesso soggette a problemi di sincronizzazione. TestFX fornisce diversi meccanismi che aiutano a coordinare le verifiche con l'effettivo aggiornamento dell'interfaccia. L'obiettivo è evitare che un test tenti di verificare uno stato prima che l'applicazione abbia completato le operazioni necessarie.

La corretta gestione della sincronizzazione è uno degli aspetti fondamentali per ridurre la presenza di flaky tests.

4) Limiti di TestFX

Nonostante i numerosi vantaggi, TestFX presenta alcune limitazioni. Innanzitutto, il framework è strettamente legato a JavaFX e non può essere utilizzato per altre tecnologie. Inoltre, come tutti gli strumenti di GUI Testing, non elimina completamente problemi quali:

- fragilità dei test;
- costi di manutenzione;
- gestione dell'asincronia;
- difficoltà nella definizione degli oracoli di test.

Di conseguenza, TestFX deve essere considerato come uno strumento utile per affrontare il problema del GUI Testing, ma non come una soluzione definitiva a tutte le sue criticità.

VIII. Caso di studio: applicazione Todo List

Per valutare concretamente le problematiche e le tecniche del GUI Testing è stata sviluppata una semplice applicazione desktop denominata *Todo List*. L'obiettivo dell'applicazione non è la realizzazione di un prodotto complesso, ma la costruzione di un ambiente sufficientemente realistico da consentire la sperimentazione di una delle principali tecniche di testing automatico delle interfacce grafiche.

L'applicazione è stata sviluppata utilizzando Scala e JavaFX, mentre il testing automatico è stato realizzato attraverso il framework TestFX.

La scelta di una Todo List è motivata dalla presenza di numerose interazioni tipiche delle moderne interfacce grafiche, tra cui:

- inserimento di dati;
- selezione di elementi;
- aggiornamento dinamico della GUI;
- gestione dello stato;
- utilizzo di finestre di dialogo;
- persistenza dei dati.

Tali caratteristiche la rendono un caso di studio adeguato per l'analisi del GUI Testing.

1) Requisiti funzionali

L'applicazione implementa le seguenti funzionalità:

- **Inserimento di attività:** L'utente può aggiungere una nuova attività attraverso un campo di testo e un pulsante dedicato.
- **Validazione dell'input:** Il sistema impedisce l'inserimento di attività vuote e segnala l'errore mediante un messaggio informativo.
- **Completamento delle attività:** Ogni attività può essere marcata come completata attraverso una checkbox.
- **Eliminazione delle attività:** L'utente può eliminare un'attività selezionata. Prima della cancellazione viene richiesta una conferma tramite una finestra di dialogo.
- **Filtri:** L'applicazione consente di visualizzare:
 - tutte le attività;
 - solo le attività da completare;
 - solo le attività completate.
- **Statistiche:** Viene mostrato un contatore che riporta:
 - numero totale delle attività;
 - numero attività da completare;
 - numero attività completate.

- **Persistenza:** Le attività vengono salvate automaticamente su file e ripristinate all'avvio dell'applicazione.

2) Requisiti non funzionali

Durante la progettazione sono stati considerati alcuni requisiti non funzionali particolarmente rilevanti per il testing.

- **Testabilità:** L'applicazione è stata progettata in modo da facilitare la realizzazione di test automatici. A tal fine ogni componente significativo della GUI possiede un identificatore univoco utilizzabile da TestFX.
- **Modularità:** Le responsabilità sono state separate attraverso l'adozione del pattern MVC.
- **Manutenibilità:** La struttura del progetto è stata organizzata in componenti indipendenti e facilmente modificabili.

3) Build automation

Per la gestione della compilazione è stato utilizzato **SBT (Simple Build Tool)**, lo strumento di build standard dell'ecosistema Scala.

Attraverso SBT è stato possibile automatizzare:

- compilazione del codice sorgente (sbt compile);
- esecuzione dei test (sbt test);
- gestione delle dipendenze (sbt run);
- creazione del fat JAR eseguibile (sbt assembly).

IX. Implementazione della suite di test e analisi dei risultati

La suite di test sviluppata per il progetto ha lo scopo di verificare automaticamente il corretto funzionamento delle principali funzionalità dell'applicazione Todo List.

Sono stati utilizzati due differenti livelli di testing: il **Unit Testing**, on mediante JUnit 5, per verificare il corretto funzionamento della logica applicativa ed il **GUI Testing**, mediante TestFX, per simulare l'interazione dell'utente con l'interfaccia grafica.

In particolare, la suite copre:

- inserimento di nuove attività;
- validazione degli input;
- gestione dello stato di completamento;
- eliminazione delle attività;
- applicazione dei filtri;
- aggiornamento delle statistiche.

L'obiettivo principale della sperimentazione era valutare la possibilità di automatizzare il testing di una applicazione con interfaccia grafica attraverso l'utilizzo del framework TestFX. La suite di test sviluppata è stata applicata all'applicazione Todo List realizzata in JavaFX e ha consentito di verificare automaticamente le principali funzionalità offerte dal sistema.

L'analisi dei risultati permette di valutare non soltanto la correttezza dell'applicazione sviluppata, ma anche l'efficacia dell'approccio adottato per il GUI Testing.

L'esecuzione automatica dei test ha consentito di controllare sistematicamente il comportamento dell'applicazione in presenza delle principali interazioni utente-sistema e i risultati ottenuti hanno confermato la correttezza delle funzionalità implementate.

Difficoltà incontrate

Nonostante i risultati positivi, durante la sperimentazione sono emerse alcune difficoltà tipiche del GUI Testing.

✓ **Gestione dello stato**

Molti test dipendono dallo stato corrente dell'applicazione. È stato quindi necessario garantire che ogni test venisse eseguito in condizioni controllate e indipendenti.

✓ **Persistenza dei dati**

La presenza del salvataggio su file ha introdotto la necessità di ripristinare lo stato iniziale prima dell'esecuzione dei test. In caso contrario, l'esito di un test avrebbe potuto influenzare quelli successivi.

✓ **Finestre modali**

Le finestre di dialogo richiedono una gestione specifica durante il testing e rappresentano uno degli aspetti più delicati dell'automazione delle GUI.

X. Limiti e sviluppi futuri

Nonostante i risultati positivi ottenuti durante la sperimentazione, il lavoro svolto presenta alcune limitazioni che devono essere considerate nell'interpretazione dei risultati. Come avviene in molti progetti sperimentali, il caso di studio sviluppato rappresenta una semplificazione di scenari reali generalmente più complessi.

L'analisi di tali limitazioni consente non solo di valutare correttamente i risultati ottenuti, ma anche di individuare possibili direzioni per futuri approfondimenti.

1) Limitazioni del caso di studio

a. Dimensione dell'applicazione

L'applicazione Todo List sviluppata nel progetto possiede una complessità relativamente contenuta. Pur includendo diverse tipologie di interazione tipiche delle moderne interfacce grafiche, non presenta alcune caratteristiche frequentemente presenti nei sistemi software reali, quali:

- navigazione tra numerose schermate;
- gestione avanzata degli utenti;
- accesso a servizi remoti;
- aggiornamenti concorrenti;
- grandi quantità di dati.

Di conseguenza, i risultati ottenuti devono essere interpretati nel contesto di un'applicazione di dimensioni limitate.

b. Numero di scenari testati

La suite sviluppata copre le principali funzionalità dell'applicazione, ma non esaurisce tutte le possibili sequenze di interazione. Come discusso nei capitoli precedenti, il numero di combinazioni possibili cresce rapidamente all'aumentare della complessità dell'interfaccia.

Per questo motivo è stato necessario selezionare un insieme rappresentativo di scenari ritenuti particolarmente significativi.

c. Costi di manutenzione

La manutenzione delle suite di test rappresenta uno dei principali costi associati all'automazione.

Modifiche dell'interfaccia possono richiedere aggiornamenti dei casi di test e aumentare il costo complessivo del processo di verifica.

d. Fragilità dei test

Nonostante l'utilizzo di identificatori stabili e di buone pratiche progettuali, i test GUI possono risultare più fragili rispetto agli unit test.

Questa caratteristica costituisce una delle principali sfide ancora aperte nel settore.

2) Possibili sviluppi futuri

Il progetto potrebbe essere esteso in diverse direzioni.

a. Incremento della complessità applicativa

Una prima evoluzione potrebbe consistere nell'ampliamento della Todo List con funzionalità aggiuntive, ad esempio:

- categorie;
- priorità delle attività;
- scadenze;
- ricerca;

L'aumento della complessità consentirebbe di analizzare scenari di testing più articolati.

b. Property-Based Testing

Un ulteriore sviluppo potrebbe riguardare l'applicazione di tecniche di Property-Based Testing. In questo caso non verrebbero definiti singoli casi di test, ma proprietà generali che il sistema dovrebbe sempre rispettare.

L'approccio consentirebbe di aumentare il grado di automazione e la capacità di individuare comportamenti inattesi.

c. Studio dei Flaky Tests

La gestione dei flaky tests rappresenta ancora oggi un tema di ricerca attivo. Un approfondimento futuro potrebbe concentrarsi sull'analisi delle cause che generano instabilità nei test GUI e sulle strategie per ridurre l'impatto.

XI. Retrospettiva

Lo sviluppo dell'applicazione ha permesso di mettere in pratica numerosi concetti affrontati durante il corso di Paradigmi di Programmazione, sia dal punto di vista della progettazione software sia da quello dell'utilizzo del linguaggio Scala. Fin dalle prime fasi del progetto è stata adottata un'architettura Model–View–Controller (MVC), che ha consentito di separare chiaramente la logica di business dall'interfaccia grafica e dalla gestione della persistenza. Questa scelta si è dimostrata particolarmente efficace durante le fasi di testing e manutenzione, poiché ha permesso di modificare singole componenti senza introdurre cambiamenti nelle altre parti del sistema.

L'utilizzo di Scala 3 ha inoltre rappresentato un'opportunità per approfondire alcuni aspetti della programmazione funzionale, come l'uso di strutture dati immutabili, del pattern matching e del tipo Either per la gestione degli errori.

Durante lo sviluppo sono emerse alcune difficoltà, principalmente legate alla configurazione dell'ambiente di sviluppo e degli strumenti utilizzati. Una delle problematiche principali ha riguardato la configurazione di JavaFX con SBT, in particolare per quanto riguarda la corretta gestione delle dipendenze e la generazione di un artefatto eseguibile contenente tutte le librerie necessarie. Anche la configurazione dei test automatici della GUI ha richiesto particolare attenzione. L'utilizzo di TestFX ha reso necessario assegnare identificatori univoci ai componenti grafici e comprendere il funzionamento del thread grafico di JavaFX. Durante lo sviluppo sono stati inoltre risolti alcuni problemi relativi alla selezione degli elementi della lista e alla gestione delle finestre di dialogo di conferma. Infine, la generazione del fat JAR tramite sbt-assembly ha richiesto la definizione di specifiche strategie di merge per risolvere i conflitti tra alcune risorse delle librerie JavaFX.

XII. Conclusioni

L'obiettivo di questo progetto era analizzare come sia possibile automatizzare il testing delle interfacce grafiche, approfondendo le principali problematiche associate a questo processo e valutando l'utilizzo di strumenti specifici per affrontarle.

A tal fine è stato innanzitutto introdotto il contesto generale del Software Testing, evidenziando il ruolo del GUI Testing all'interno delle attività di verifica e validazione del software. Successivamente sono state analizzate le caratteristiche che rendono il testing delle interfacce grafiche particolarmente complesso rispetto ad altre forme di testing. In particolare sono stati discussi aspetti quali:

- la natura event-driven delle applicazioni grafiche;
- la gestione dello stato dell'interfaccia;
- l'esplosione combinatoria delle possibili interazioni;
- i problemi di sincronizzazione;
- il Test Oracle Problem;
- la presenza di flaky tests;
- i costi di manutenzione delle suite di test.

Sono state inoltre esaminate le principali tecniche di automazione del GUI Testing e i framework maggiormente diffusi, confrontandone caratteristiche, vantaggi e limitazioni. Tra le diverse soluzioni disponibili è stato scelto il framework TestFX, particolarmente adatto al testing di applicazioni sviluppate con JavaFX.

Per valutare concretamente l'approccio proposto è stata sviluppata una applicazione Todo List realizzata secondo il pattern architetturale Model-View-Controller e dotata di diverse funzionalità interattive. Su tale applicazione è stata progettata e implementata una suite di test automatici finalizzata a verificare il corretto comportamento dell'interfaccia grafica.

La domanda che ha guidato il progetto può essere formulata come segue:

Com'è possibile fare test automatizzato di interfacce grafiche?

L'analisi svolta e la sperimentazione realizzata permettono di affermare che il testing automatizzato delle interfacce grafiche è possibile attraverso strumenti in grado di simulare le interazioni dell'utente e verificare automaticamente lo stato dell'applicazione.

Framework come TestFX consentono di:

- individuare componenti dell'interfaccia;
- simulare click e inserimenti di testo;
- interagire con finestre e controlli grafici;
- verificare automaticamente il comportamento osservabile del sistema.

In questo modo è possibile riprodurre in maniera controllata e ripetibile numerosi scenari di utilizzo normalmente eseguiti dagli utenti.

La sperimentazione condotta ha mostrato che l'automazione del GUI Testing offre diversi vantaggi. Innanzitutto consente di ridurre il tempo necessario per l'esecuzione delle verifiche e di aumentare la ripetibilità dei test. Inoltre permette di individuare rapidamente regressioni introdotte durante l'evoluzione del software e di integrare le attività di verifica nei moderni processi di sviluppo continuo. Dal punto di vista pratico, TestFX si è dimostrato uno strumento efficace e relativamente semplice da utilizzare per il testing di applicazioni JavaFX.

Nonostante i benefici osservati, il progetto ha confermato che il GUI Testing automatizzato non rappresenta una soluzione priva di criticità. Inoltre, aspetti quali usabilità, accessibilità ed esperienza utente risultano difficilmente valutabili attraverso procedure completamente automatizzate. Per queste ragioni il testing automatico deve essere considerato complementare e non sostitutivo rispetto alle attività di testing manuale.

Il lavoro svolto ha evidenziato come il GUI Testing rappresenti un ambito particolarmente interessante all'interno del Software Testing, caratterizzato da sfide specifiche ma anche da importanti opportunità.

L'esperienza maturata attraverso lo sviluppo dell'applicazione Todo List e della relativa suite di test ha dimostrato che, mediante una corretta progettazione dell'architettura software e l'utilizzo di framework adeguati, è possibile realizzare processi di GUI Testing efficaci, affidabili e facilmente integrabili nel ciclo di sviluppo.

In conclusione, il testing automatizzato delle interfacce grafiche costituisce oggi uno strumento fondamentale per migliorare la qualità del software e rappresenta un ambito di ricerca e applicazione destinato ad assumere un ruolo sempre più rilevante nello sviluppo delle applicazioni moderne.